

Лабораторная работа №7

Введение в работу с данными

Дурдалыев Максат

Содержание

1	Введение	6
1.1	Цели и задачи	6
2	Выполнение лабораторной работы	7
2.1	Julia для науки о данных	7
2.2	Считывание данных	7
2.3	Запись данных в файл	19
2.4	Словари	25
2.5	DataFrames	33
2.6	RDatasets	37
2.7	Работа с переменными отсутствующего типа (Missing Values)	41
2.8	FileIO	49
2.9	Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних	51
2.10	Кластеризация данных. Метод k ближайших соседей	64
2.11	Обработка данных. Метод главных компонент	68
2.12	Обработка данных. Линейная регрессия	70
2.13	Самостоятельное выполнение	77
3	Вывод	82
	Список литературы	83

Список иллюстраций

2.1	Установка пакетов	8
2.2	Считывание данных и запись в структуру	10
2.3	Пример	12
2.4	Поиск «julia» со строчной буквы	14
2.5	Изменение исходной функции	16
2.6	Построчное считывание данных	18
2.7	Запись данных в файл	20
2.8	Пример с указанием типа данных и разделителем данных	22
2.9	Проверка корректности считывания созданного текстового файла	24
2.10	Инициализация словаря	26
2.11	Инициализация пустого словаря	28
2.12	Заполнение словаря данными	30
2.13	Пример работы словаря	32
2.14	Пример создания структуры DataFrame	34
2.15	Пример создания структуры DataFrame	36
2.16	Работа с пакетом RDatasets	38
2.17	Получение основных статических сведений о каждом столбце в наборе данных	40
2.18	Использование «отсутствующего» типа. Операция сложения числа и переменной с отсутствующим типом	42
2.19	Пример работы с данными, среди которых есть данные с отсутствующим типом	44
2.20	Игнорирование отсутствующего типа	46
2.21	Формирование таблиц данных и их объединение в один фрейм	48
2.22	Подключение пакетов. Загрузка изображения	50
2.23	Загрузка данных	52
2.24	Построение графика цен на недвижимость в зависимости от площади	54
2.25	Построение графика без «артефактов»	56
2.26	Среднее значение	58
2.27	Построение графика с кластерами разных цветов	60
2.28	Построение графика с кластерами разных цветов	61
2.29	Построение графика с кластерами разных цветов по почтовому индексу	63
2.30	Отображение на графике соседей выбранного объекта недвижимости	65
2.31	Определение районов соседних домов	67

2.32	Попытка уменьшения размера данных о цене и площади из набора данных домов	69
2.33	Исходные данные	71
2.34	Применение функции для построения графика	73
2.35	Генерация данных. Определние времени соответствия	75
2.36	Сравнение	76
2.37	Кластеризация	77
2.38	Кластеризация	77
2.39	Кластеризация	78
2.40	Регрессия	78
2.41	Регрессия	79
2.42	Регрессия	79
2.43	Модель ценообразования биномиальных опционов	80
2.44	Модель ценообразования биномиальных опционов	80
2.45	Модель ценообразования биномиальных опционов	81
2.46	Модель ценообразования биномиальных опционов	81
2.47	Модель ценообразования биномиальных опционов	81
2.48	Модель ценообразования биномиальных опционов	81

Список таблиц

1 Введение

1.1 Цели и задачи

Цель работы

Основной целью работы является специализированных пакетов Julia для обработки данных[2].

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы[1].

2 Выполнение лабораторной работы

2.1 Julia для науки о данных

В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python.

2.2 Считывание данных

Перед тем, как начать проводить какие-либо операции над данными, необходимо их откуда-то считать и возможно сохранить в определённой структуре.

Довольно часто данные для обработки содержатся в csv-файле, имеющим текстовый формат, в котором данные в строке разделены, например, запятыми, и соответствуют ячейкам таблицы, а строки данных соответствуют строкам таблицы. Также данные могут быть представлены в виде фреймов или множеств.

В Julia для работы с такого рода структурами данных используют пакеты CSV, DataFrames, RDatasets, FileIO (рис. [fig:001]):

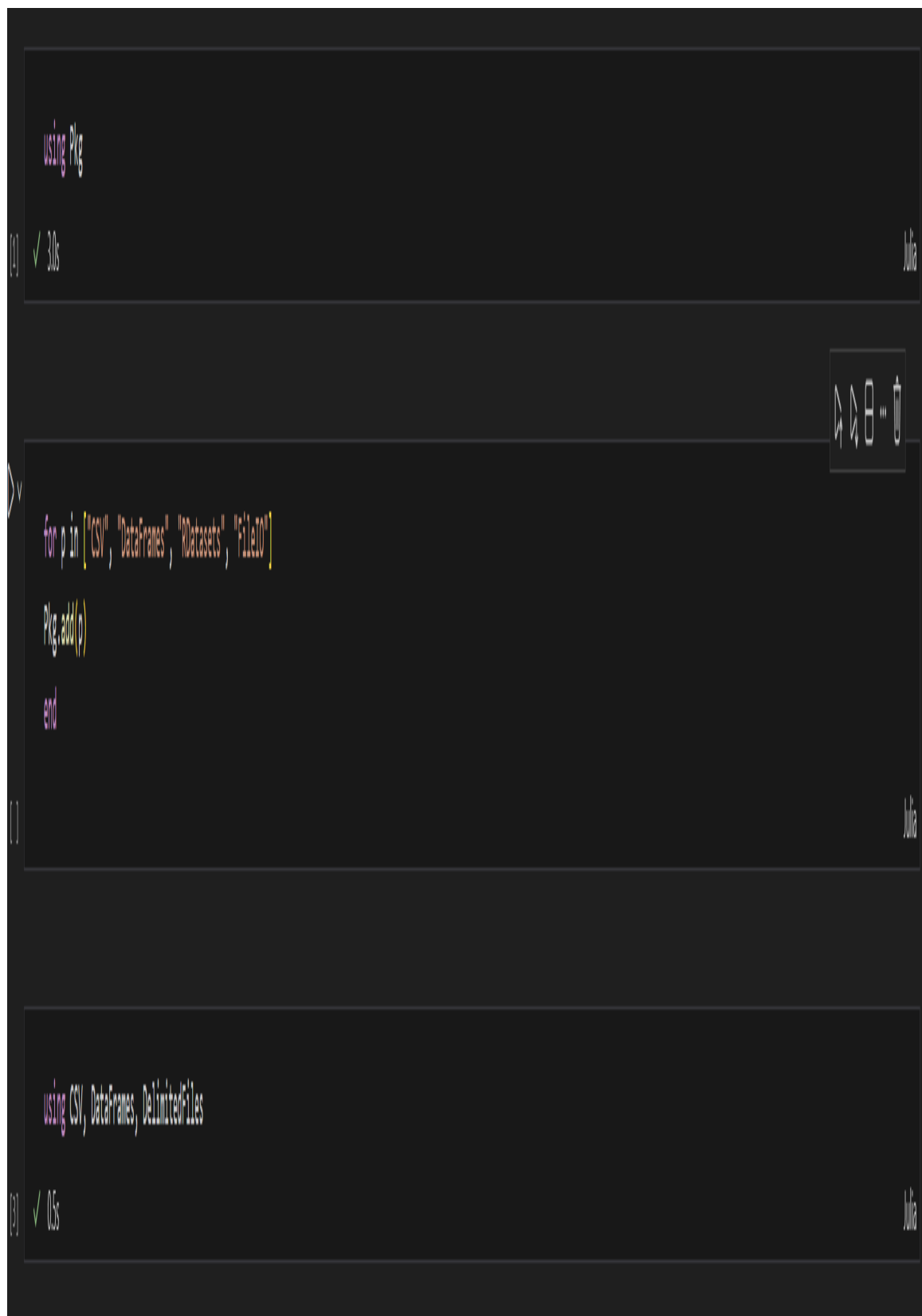


Рисунок 2.1: Установка пакетов

Предположим, что у нас в рабочем каталоге с проектом есть файл с данными `programminglanguages.csv`, содержащий перечень языков программирования и год их создания. Тогда для заполнения массива данными для последующей обработки требуется считать данные из исходного файла и записать их в соответствующую структуру (рис. [fig:002]):

```
# Считывание данных и их запись в структуру:
P = CSV.File("programminglanguages.csv") |> DataFrame

# Функция определения по названию языка программирования года его создания:
function language_created_year(P::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

✓ 01s

language_created_year (generic function with 1 method)

# Пример вызова функции и определение даты создания языка Python:
language_created_year(P,"Python")

✓ 02s

1991
```

Рисунок 2.2: Считывание данных и запись в структуру

Пример для Julia (рис. [fig:003]):

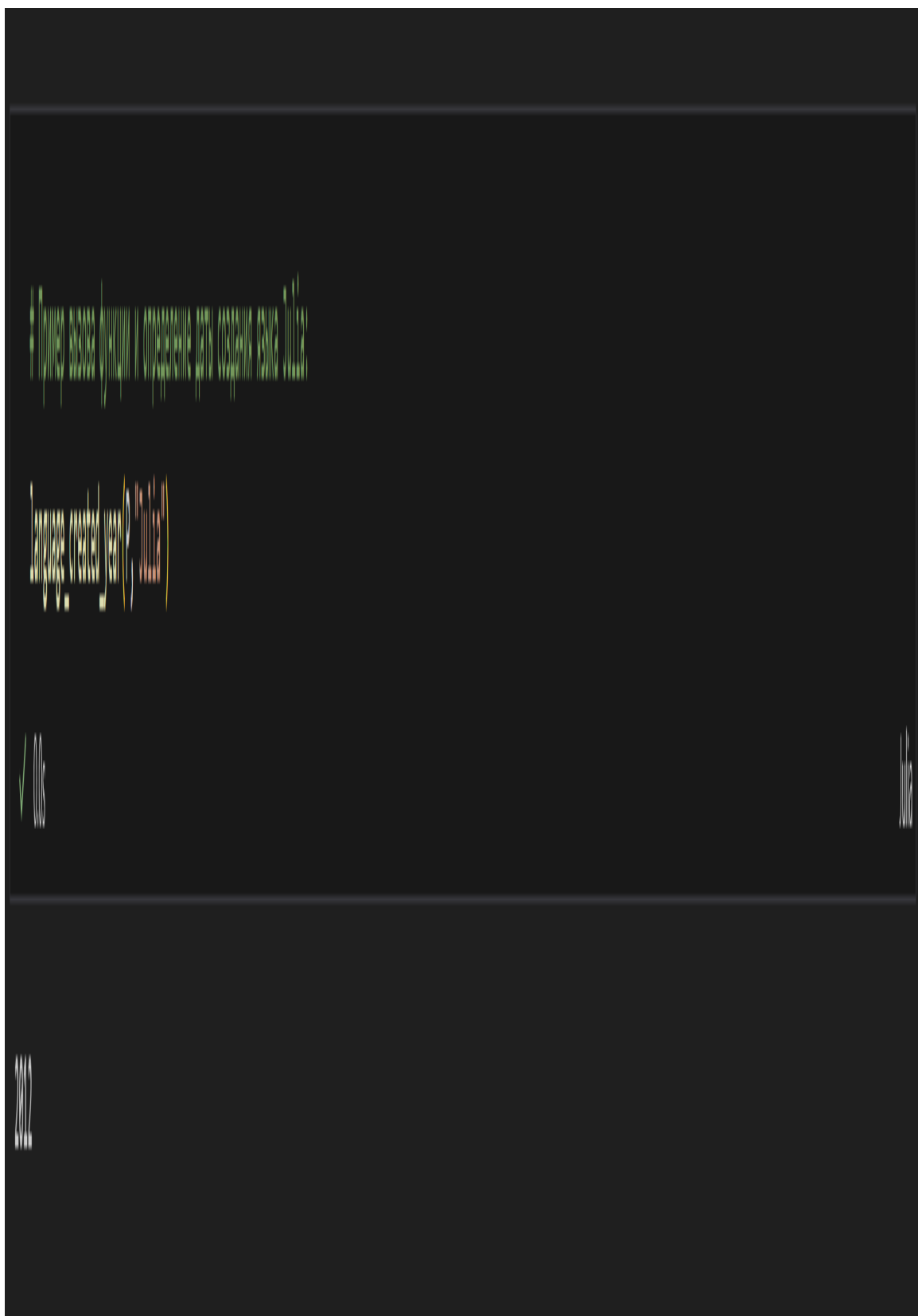


Рисунок 2.3: Пример

В следующем примере при вызове функции, в качестве аргумента которой указано слово `julia`, написанное со строчной буквы (рис. [fig:004]):

```
language_created_year(P, "julia")

ⓧ 1.1s Julia

MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)
The function `getindex` exists, but no method is defined for this combination of argument types.

Closest candidates are:
  getindex(::DataFrame, !Matched::typeof(!), ::Union{Signed, Unsigned})
    @ DataFrames C:\Users\durda\julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:548
  getindex(::DataFrame, !Matched::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames C:\Users\durda\julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:542
  getindex(::DataFrame, !Matched::Integer, ::Union{Signed, Unsigned})
    @ DataFrames C:\Users\durda\julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:506
  ...

Stacktrace:
 [1] language_created_year(P::DataFrame, language::String)
    @ Main d:\V\4\4 kypc\Computer practice\Ia6a 7\jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X24sZmlsZQ==.jl:4
 [2] top-level scope
    @ d:\V\4\4 kypc\Computer practice\Ia6a 7\jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X21sZmlsZQ==.jl:1
```

Рисунок 2.4: Поиск «julia» со строчной буквы

Для того, чтобы убрать в функции зависимость данных от регистра, необходимо изменить исходную функцию следующим образом (рис. [fig:005]):

```
# Функция определения по названию языка программирования
# года его создания (без учёта регистра):

function language_created_year_v2(P::String)

    loc = findfirst(lowercase(P[1:2]) == lowercase(language))

    return P[loc,1]

end

✓ 0.0s Julia

language_created_year_v2 (generic function with 1 method)

# Пример вызова функции и определение даты создания языка Julia:

language_created_year_v2(P,"Julia")

✓ 0.1s Julia

2012
```

Рисунок 2.5: Изменение исходной функции

Можно считывать данные построчно, с элементами, разделенными заданным разделителем (рис. [fig:006]):

```
# Построчное считывание данных с указанием разделителя:
Tx = readlm("programminglanguages.csv", ',')

✓ 16s Julia

74x2 Matrix{Any}:
  "year"  "Language"
1951  "Regional Assembly Language"
1952  "Autocode"
1954  "IPL"
1955  "FLOW-MATIC"
1957  "FORTRAN"
1957  "CONTRAN"
1958  "LISP"
1958  "ALGOL 58"
1959  "FACT"
  ⋮
2007  "Clojure"
2009  "Go"
2010  "Rust"
2011  "Dart"
2011  "Kotlin"
2011  "Red"
2011  "Elixir"
2012  "Julia"
2014  "Shift"
```

Рисунок 2.6: Построчное считывание данных

2.3 Запись данных в файл

Предположим, что требуется записать имеющиеся данные в файл. Для записи данных в формате CSV можно воспользоваться следующим вызовом (рис. [fig:007]):

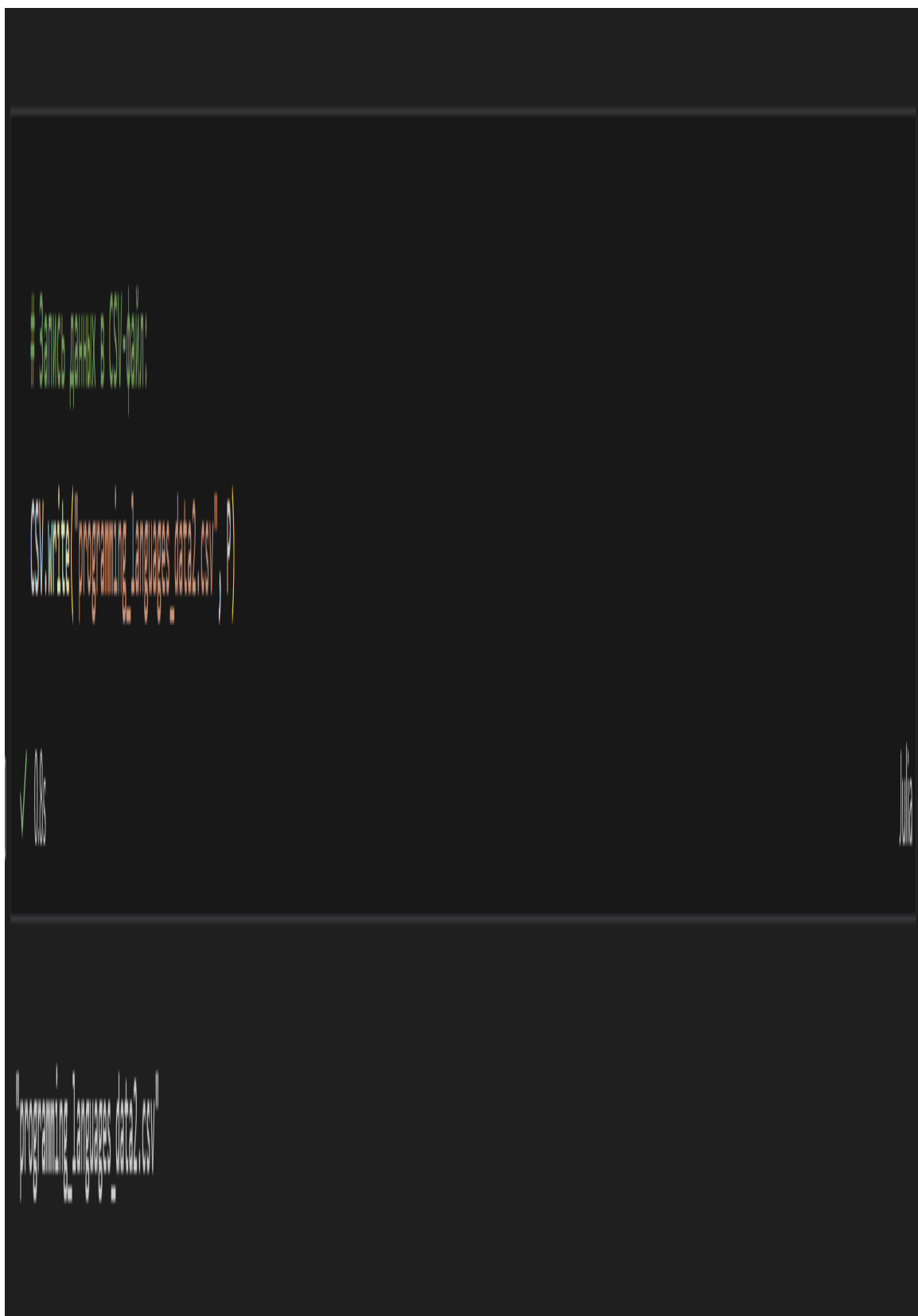


Рисунок 2.7: Запись данных в файл

Можно задать тип файла и разделитель данных (рис. [fig:008]):

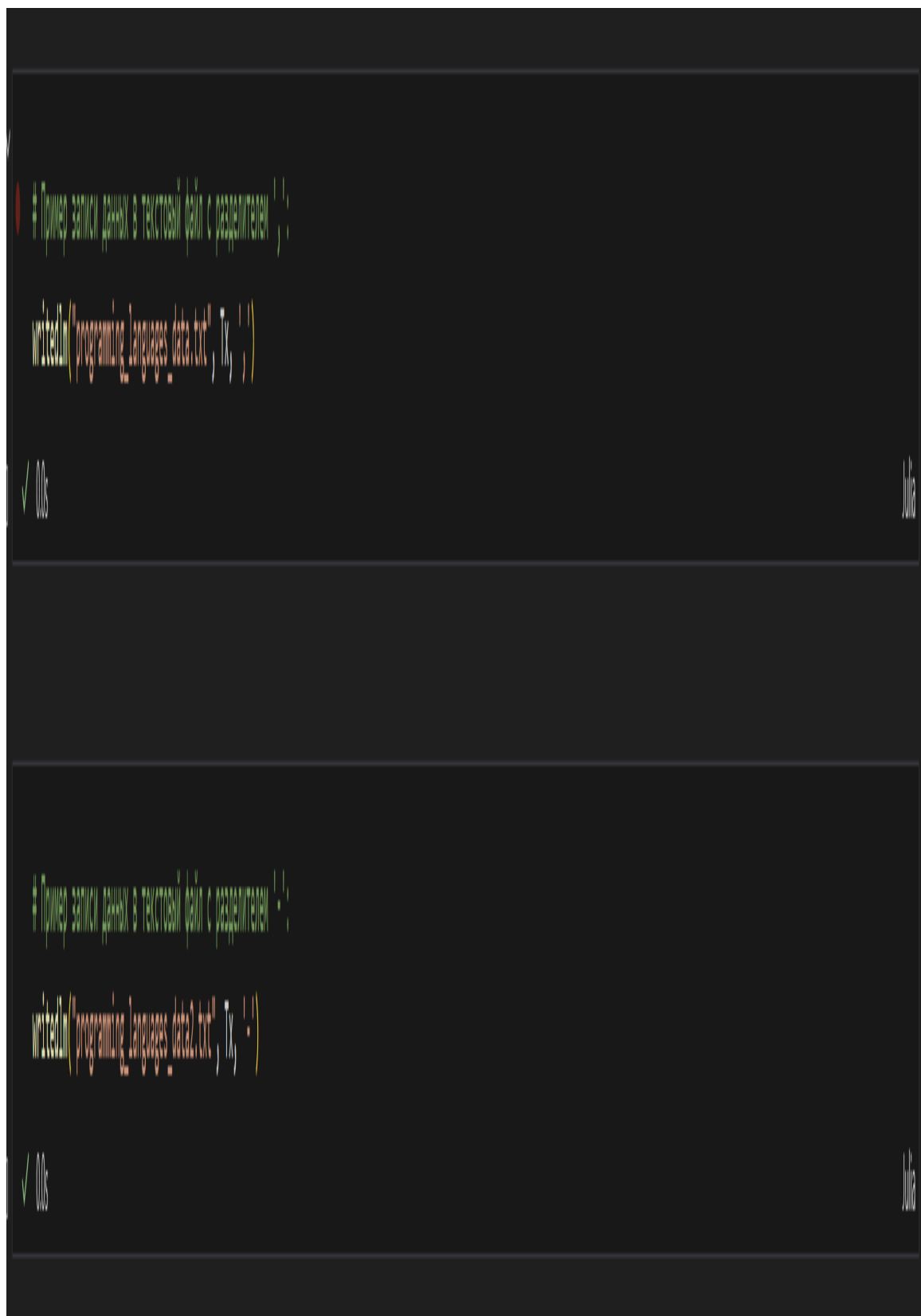


Рисунок 2.8: Пример с указанием типа данных и разделителем данных

Можно проверить, используя `readrlm`, корректность считывания созданного текстового файла (рис. [fig:009]):

```
# Построчное считывание данных с указанием разделителя:
P_new_delim = readlm("programming_languages_data2.txt", '-')

✓ 0.0s Julia
```

74x2 Matrix{Any}:

"year"	"language"
1951	"Regional Assembly Language"
1952	"Autocode"
1954	"IPL"
1955	"FLOW-MATIC"
1957	"FORTRAN"
1957	"CONTRAN"
1958	"LISP"
1958	"ALGOL 58"
1959	"FACT"
⋮	
2007	"Clojure"
2009	"Go"
2010	"Rust"
2011	"Dart"
2011	"Kotlin"
2011	"Red"
2011	"Elixir"
2012	"Julia"
2014	"Swift"

Рисунок 2.9: Проверка корректности считывания созданного текстового файла

2.4 Словари

При работе с данными бывает удобно записать их в формате словаря.

Предположим, что словарь должен содержать перечень всех языков программирования и года их создания, при этом при указании года выводить все языки программирования, созданные в этом году.

При инициализации словаря можно задать конкретные типы данных для ключей и значений (рис. [fig:010]):

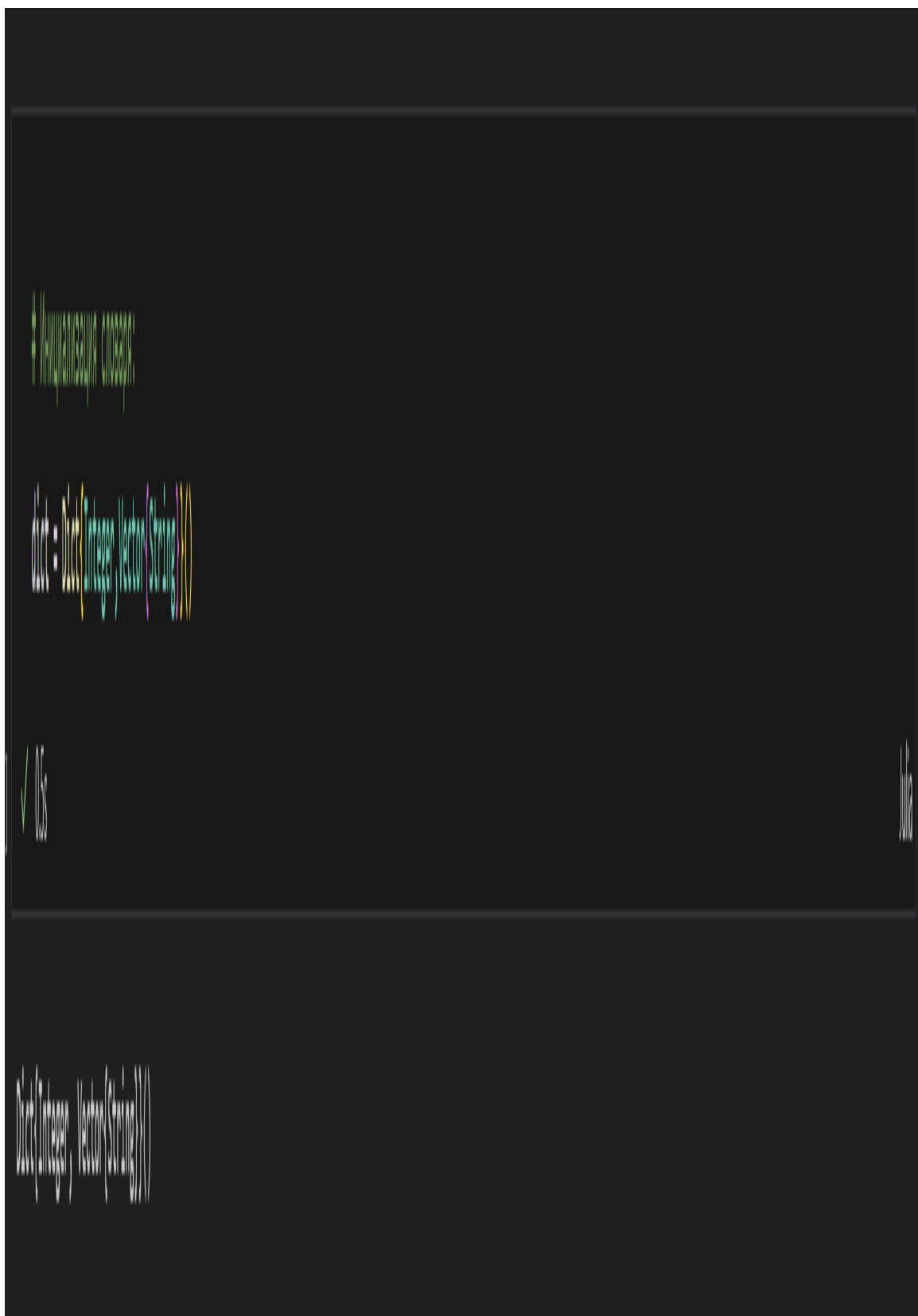


Рисунок 2.10: Инициализация словаря

Можно инициировать пустой словарь, не задавая строго структуру (рис. [fig:011]):

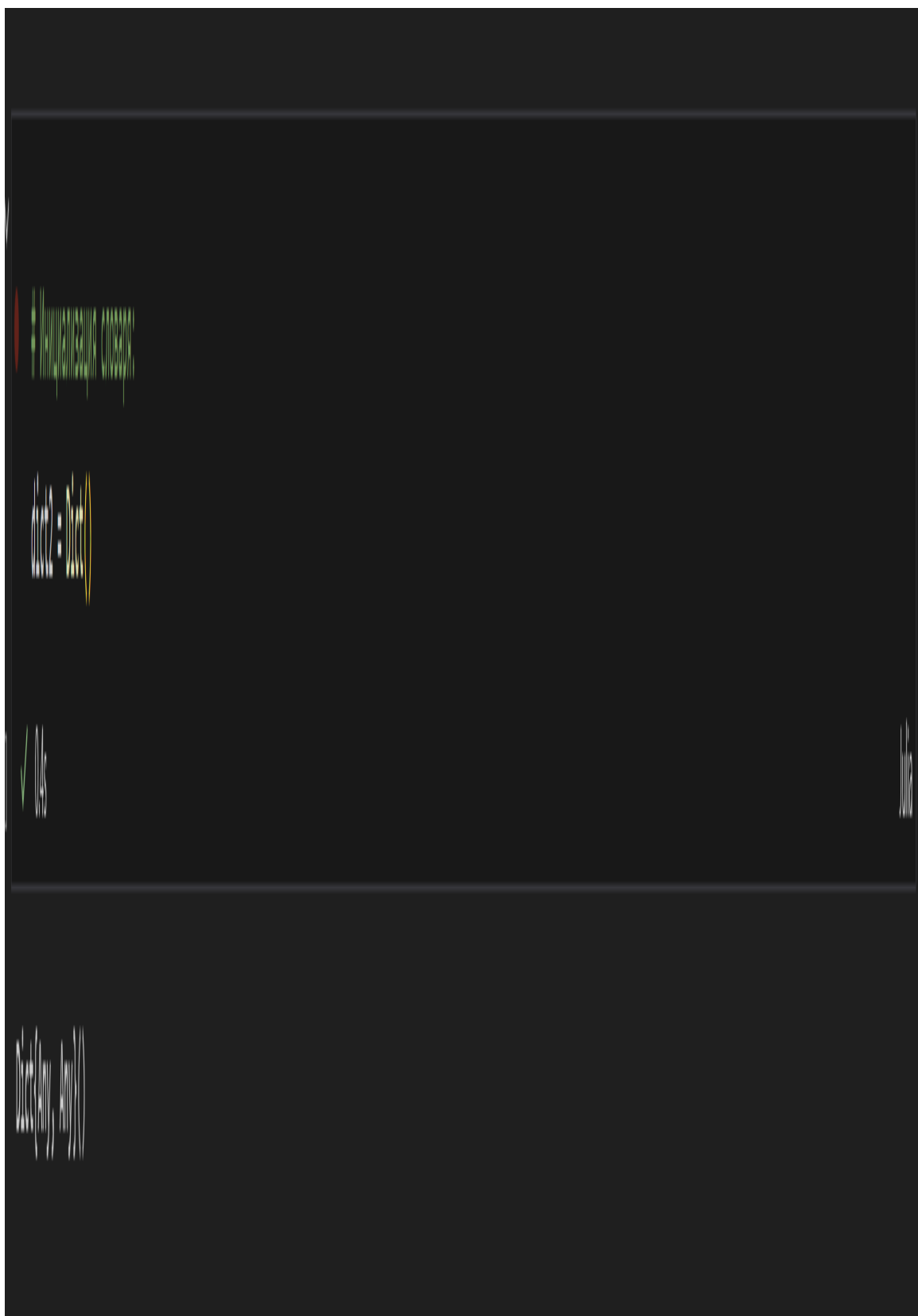


Рисунок 2.11: Инициализация пустого словаря

Далее требуется заполнить словарь ключами и годами, которые содержат все языки программирования, созданные в каждом году, в качестве значений (рис. [fig:012]):

```
# Заполнение словаря данными:  
  
for i = 1:size(P,1)  
    year_lang = P[i,:]  
    if year in keys(dict)  
        dict[year] = push!(dict[year], lang)  
    else  
        dict[year] = [lang]  
    end  
end  
  
✓ @ts
```

julia

Рисунок 2.12: Заполнение словаря данными

В результате при вызове словаря можно, выбрав любой год, узнать, какие языки программирования были созданы в этом году (рис. [fig:013]):

June

"Scala"

2.5 DataFrames

Работа с данными, записанными в структуре DataFrame, позволяет использовать индексацию и получить доступ к столбцам по заданному имени заголовка или по индексу столбца.

На примере с данными о языках программирования и годах их создания зададим структуру DataFrame (рис. [fig:014]):

```
# Задаём переменную со структурой DataFrame:
df = DataFrame(year = P[:,1], language = P[:,2])
```

✓ 0.0s Julia

73×2 DataFrame 48 rows omitted

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58
9	1959	FACT
10	1959	COBOL
11	1959	RPG
12	1962	APL
13	1962	Simula
:	:	:
62	2003	Scala
63	2005	F#
64	2006	PowerShell
65	2007	Clojure
66	2009	Go
67	2010	Rust
68	2011	Dart
69	2011	Kotlin

Рисунок 2.14: Пример создания структуры DataFrame

Если требуется получить доступ к столбцам по имени заголовка, то необходимо добавить к имени заголовка двоеточие. В Julia это означает, что имена заголовков обрабатываются как символы. Также следует иметь в виду, что вызов `df[1]` эквивалентен вызову `df [:year]`. Пакет DataFrames предоставляет возможность с помощью `description` получить основные статистические сведения о каждом столбце во фрейме данных(рис. [fig:015]).

```
# Вывод всех значения столбца year:
df[:,year]
```

✓ 0.2s Julia

73-element Vector{Int64}:
1951
1952
1954
1955
1957
1957
1958
1958
1959
1959
:
2007
2009
2010
2011
2011
2011
2011
2012
2014

```
# Получение статистических сведений о фрейме:
describe(df)
```

✓ 0.8s Julia

2x7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	year	1982.99	1951	1986.0	2014	0	Int64
2	language		ALGOL 58		dBase III	0	String31

Рисунок 2.15: Пример создания структуры DataFrame

2.6 RDatasets

С данными можно работать также как с наборами данных через пакет RDatasets языка R (рис. [fig:016]):

```
# Подгружаем пакет RDatasets:
using RDatasets

✓ 0.2s Julia
```

```
# Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")

✓ 6.5s Julia
```

150x5 DataFrame 125 rows omitted

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
⋮	⋮	⋮	⋮	⋮	⋮
139	6.0	3.0	4.8	1.8	virginica
140	6.9	3.1	5.4	2.1	virginica
141	6.7	3.1	5.6	2.4	virginica
142	6.9	3.1	5.1	2.3	virginica
143	5.8	2.7	5.1	1.9	virginica
144	6.8	3.2	5.9	2.3	virginica
145	6.7	3.3	6.7	2.6	virginica

Рисунок 2.16: Работа с пакетом RDatasets

Пакет `RDatasets` также предоставляет возможность с помощью `description` получить основные статистические сведения о каждом столбце в наборе данных (рис. [fig:017]):

```
# Определения типа переменной:
typeof(iris)

✓ 0.2s Julia
```

DataFrame

```
describe(iris)

✓ 0.5s Julia
```

5x7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species	setosa	virginica			0	CategoricalValue{String, UInt8}

Рисунок 2.17: Получение основных статических сведений о каждом столбце в наборе данных

2.7 Работа с переменными отсутствующего типа (Missing Values)

Пакет DataFrames позволяет использовать так называемый «отсутствующий» тип (рис. [fig:018]). В операции сложения числа и переменной с отсутствующим типом значение также будет иметь отсутствующий тип.



Рисунок 2.18: Использование «отсутствующего» типа. Операция сложения числа и переменной с отсутствующим типом

Приведём пример работы с данными, среди которых есть данные с отсутствующим типом.

Предположим есть перечень продуктов, для которых заданы калории. В массиве значений калорий есть значение с отсутствующим типом (рис. [fig:019]):

```
# Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]

# Определение калорий:
calories = [missing, 47, 22, 105]

✓ 0.3s Julia

4-element Vector{Union{Missing, Int64}}:
 missing
  47
  22
 105

# Определение типа переменной:
typeof(calories)

✓ 0.0s Julia

Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рисунок 2.19: Пример работы с данными, среди которых есть данные с отсутствующим типом

При попытке получить среднее значение калорий, ничего не получится из-за наличия переменной с отсутствующим типом.

Для решения этой проблемы необходимо игнорировать отсутствующий тип (рис. [fig:020]):

```
# Подключаем пакет Statistics:
using Statistics

✓ 0.0s Julia

# Определение среднего значения:
mean(calories)

✓ 0.1s Julia

missing

# Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))

✓ 0.1s Julia

58.0
```

Рисунок 2.20: Игнорирование отсутствующего типа

Далее показано, как можно сформировать таблицы данных и объединить их в один фрейм (рис. [fig:021]):

```

# Задание сведений о ценах:
prices = [0.85, 1.6, 0.8, 0.6]

# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods, calories=calories)

# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods, price=prices)

# Объединение данных о калориях и ценах:
Df = leftjoin(dataframe_calories, dataframe_prices, on='item')

```

✓ 29s

Julia

4x3 DataFrame

Row	item	calories	price
	String	Int64?	Float64?
1	apple	missing	0.85
2	cucumber	47	1.6
3	tomato	22	0.8
4	banana	105	0.6

Рисунок 2.21: Формирование таблиц данных и их объединение в один фрейм

2.8 FileIO

В Julia можно работать с так называемыми «сырыми» данными, используя пакет FileIO.

Попробуем посмотреть, как Julia работает с изображениями.

Подключим соответствующий пакет. Загрузим изображение (в данном случае логотип Julia)(рис. [fig:022]):



Рисунок 2.22: Подключение пакетов. Загрузка изображения

2.9 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

Задача кластеризации данных заключается в формировании однородной группы упорядоченных по какому-то признаку данных.

Метод k-средних позволяет минимизировать суммарное квадратичное отклонение точек кластеров от центров этих кластеров.

Рассмотрим задачу кластеризации данных на примере данных о недвижимости. Файл с данными `houses.csv` содержит список транзакций с недвижимостью в районе Сакраменто, о которых было сообщено в течение определённого числа дней.

Загрузим данные (рис. [fig:023]):

Загрузка данных:

houses = CSV.File("houses.csv") |> DataFrame

✓ 50s

Julia

985×12 DataFrame

960 rows omitted

Row	street	city	zip	state	beds	baths	sq_ft	type	sale_date	price	latitude	longitude
	String	String15	Int64	String3	Int64	Int64	Int64	String15	String31	Int64	Float64	Float64
1	3526 HIGH ST	SACRAMENTO	95838	CA	2	1	836	Residential	Wed May 21 00:00:00 EDT 2008	59222	38.6319	-121.435
2	51 OMAHA CT	SACRAMENTO	95823	CA	3	1	1167	Residential	Wed May 21 00:00:00 EDT 2008	68212	38.4789	-121.431
3	2796 BRANCH ST	SACRAMENTO	95815	CA	2	1	796	Residential	Wed May 21 00:00:00 EDT 2008	68880	38.6183	-121.444
4	2805 JIANETTE WAY	SACRAMENTO	95815	CA	2	1	852	Residential	Wed May 21 00:00:00 EDT 2008	69307	38.6168	-121.439
5	6001 MCMAHON DR	SACRAMENTO	95824	CA	2	1	797	Residential	Wed May 21 00:00:00 EDT 2008	81900	38.5195	-121.436
6	5828 PEPPERMILL CT	SACRAMENTO	95841	CA	3	1	1122	Condo	Wed May 21 00:00:00 EDT 2008	89921	38.6626	-121.328
7	6048 OGDEN NASH WAY	SACRAMENTO	95842	CA	3	2	1104	Residential	Wed May 21 00:00:00 EDT 2008	90895	38.6817	-121.352
8	2561 19TH AVE	SACRAMENTO	95820	CA	3	1	1177	Residential	Wed May 21 00:00:00 EDT 2008	91002	38.5351	-121.481
9	11150 TRINITY RIVER DR Unit 114	RANCHO CORDOVA	95670	CA	2	2	941	Condo	Wed May 21 00:00:00 EDT 2008	94905	38.6212	-121.271
10	7325 10TH ST	RIO LINDA	95673	CA	3	2	1146	Residential	Wed May 21 00:00:00 EDT 2008	98937	38.7009	-121.443
11	645 MORRISON AVE	SACRAMENTO	95838	CA	3	2	909	Residential	Wed May 21 00:00:00 EDT 2008	100309	38.6377	-121.452
12	4085 FAWN CIR	SACRAMENTO	95823	CA	3	2	1289	Residential	Wed May 21 00:00:00 EDT 2008	106250	38.4707	-121.459
13	2930 LA ROSA RD	SACRAMENTO	95815	CA	1	1	871	Residential	Wed May 21 00:00:00 EDT 2008	106852	38.6187	-121.436
:	:	:	:	:	:	:	:	:	:	:	:	:
974	2181 WINTERHAVEN CIR	CAMERON PARK	95682	CA	3	2	0	Residential	Thu May 15 00:00:00 EDT 2008	224500	38.6976	-120.996
975	7540 HICKORY AVE	ORANGEVALE	95662	CA	3	1	1456	Residential	Thu May 15 00:00:00 EDT 2008	225000	38.7031	-121.235
976	5024 CHAMBERLIN CIR	ELK GROVE	95757	CA	3	2	1450	Residential	Thu May 15 00:00:00 EDT 2008	228000	38.3898	-121.446
977	2400 INVERNESS DR	LINCOLN	95648	CA	3	2	1358	Residential	Thu May 15 00:00:00 EDT 2008	229027	38.8978	-121.325
978	5 BISHOPGATE CT	SACRAMENTO	95823	CA	4	2	1329	Residential	Thu May 15 00:00:00 EDT 2008	229500	38.4679	-121.445
979	5601 REKLEIGH DR	SACRAMENTO	95823	CA	4	2	1715	Residential	Thu May 15 00:00:00 EDT 2008	230000	38.4453	-121.442

Рисунок 2.23: Загрузка данных

Построим график цен на недвижимость в зависимости от площади (рис. [fig:024]):

```
# Построение графика:
plot(size=(500,500),leg=false)
x = houses[:,sq_ft]
y = houses[:,price]
scatter(x,y,markersize=3)
```

✓ 0.6s

Julia

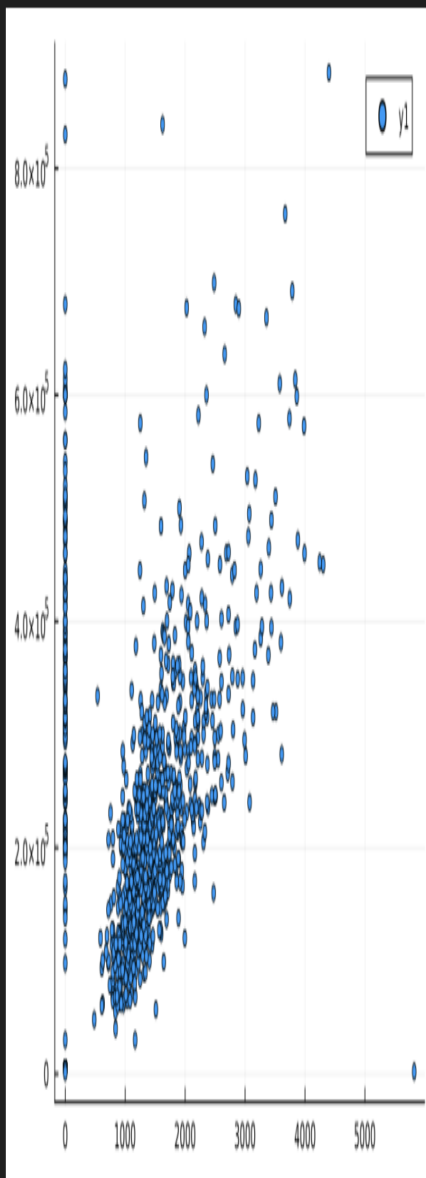


Рисунок 2.24: Построение графика цен на недвижимость в зависимости от площади

Для того чтобы избавиться от «артефактов», можно отфильтровать и исключить такие значения, получить более корректный график цен(рис. [fig:025]):

```
# Фильтрация данных по заданному условию:
```

```
filter_houses = houses[houses[:,sq_ft].>0,:]
```

```
# Построение графика:
```

```
x = filter_houses[:,sq_ft]
```

```
y = filter_houses[:,price]
```

```
scatter(x,y)
```

✓ 02s

Julia

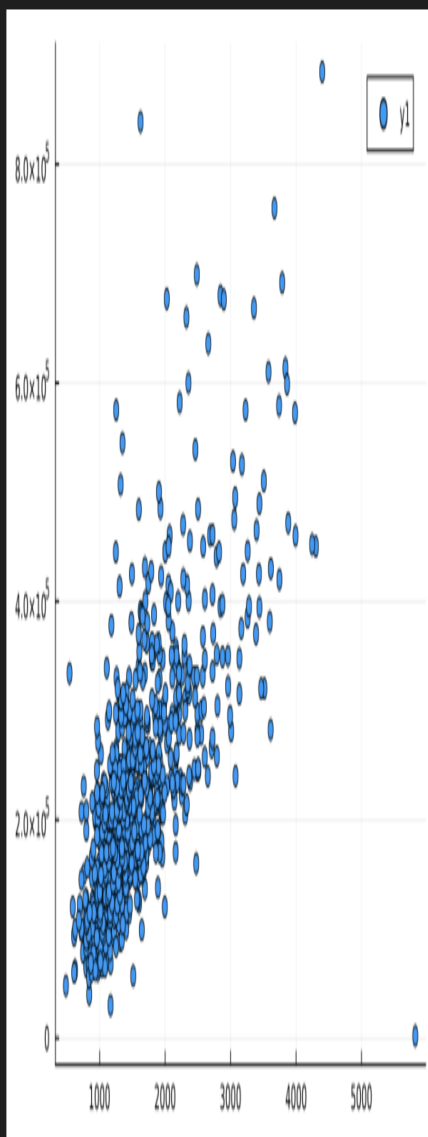


Рисунок 2.25: Построение графика без «артефактов»

Используя для фильтрации значений функцию `by` пакета `DataFrames` и для вычисления среднего значения функцию `mean` пакета `Statistics`, можно посмотреть среднюю цену домов определённого типа(рис. [fig:026]).

using Statistics

Определение средней цены для каждого типа домов

```
mean_prices = combine(groupby(filter_houses, :type), :price => mean => :mean_price)
```

```
println("Средние цены для каждого типа домов:")
```

```
println(mean_prices)
```

✓ 1.2s

Julia

Средние цены для каждого типа домов:

3x2 DataFrame

Row	type	mean_price
	String15	Float64
1	Residential	2.34802e5
2	Condo	1.34218e5
3	Multi-Family	2.24535e5

Рисунок 2.26: Среднее значение

Построим график, обозначив каждый кластер отдельным цветом (рис. [fig:027] - рис. [fig:028]):

```

# Добавление данных : latitude и : longitude
X = filter_houses[:, [:latitude, :longitude]]

# Преобразование DataFrame в матрицу
X = Matrix{Float64}(X)'

k = length(unique(filter_houses[:, :zip]))

# Определение кластеров k-среднего
C = kmeans(X, k)

# Формирование DataFrame с результатами кластеризации
df = DataFrame(
    cluster = C.assignments,
    city = filter_houses[:, :city],
    latitude = filter_houses[:, :latitude],
    longitude = filter_houses[:, :longitude],
    zip = filter_houses[:, :zip]
)

# Построение графика кластеров
clusters_figure = plot(legend=false)

for i in 1:k
    clustered_houses = df[df[:, :cluster] .== i, :]
    xvals = clustered_houses[:, :latitude]
    yvals = clustered_houses[:, :longitude]
    scatter!(clusters_figure, xvals, yvals, markersize=4)
end

xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)

```

✓ 12s

Julia

Рисунок 2.27: Построение графика с кластерами разных цветов

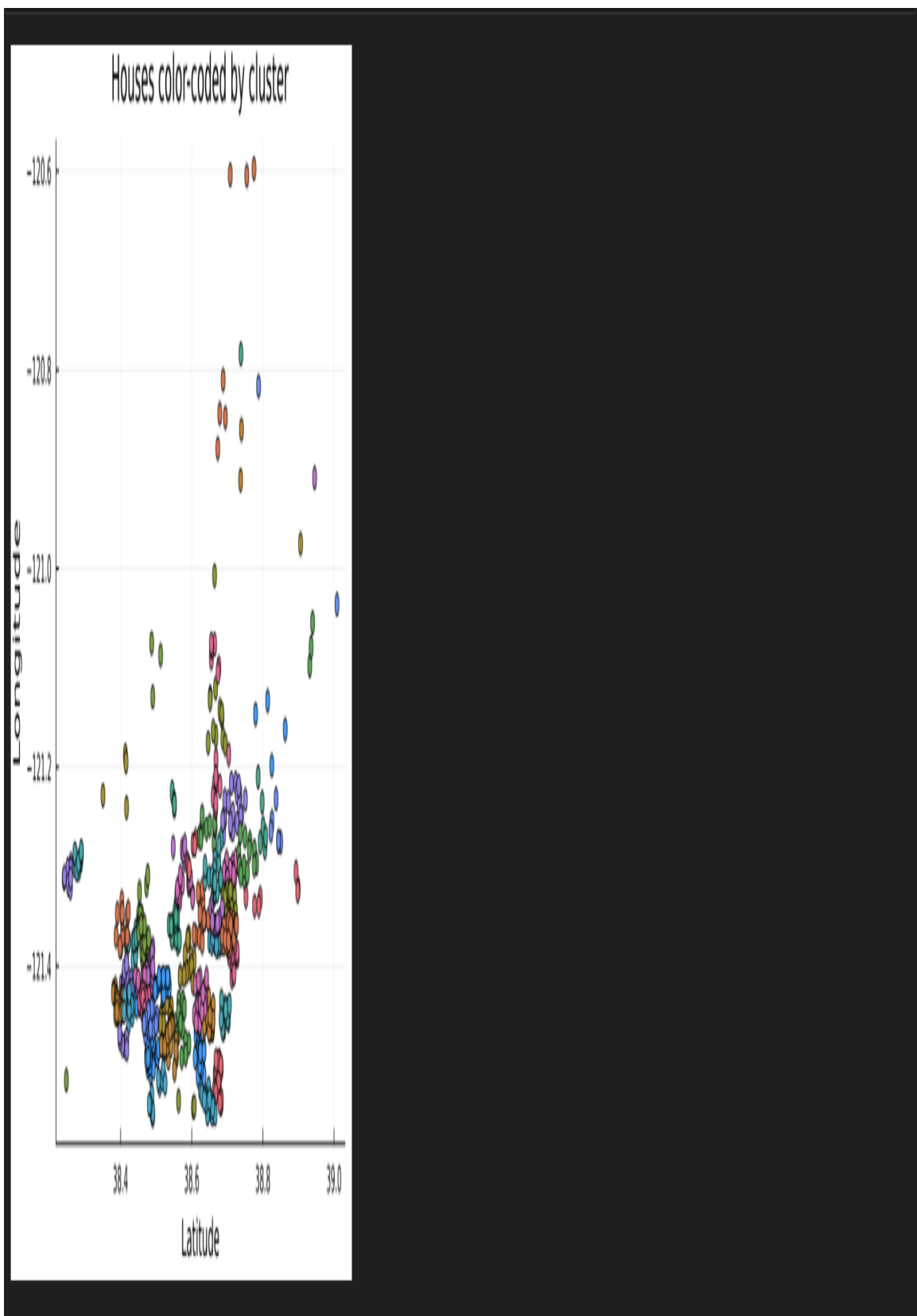


Рисунок 2.28: Построение графика с кластерами разных цветов

Построим график, раскрасив кластеры по почтовому индексу (рис. [fig:029]):

```

unique_zips = unique(filter_houses[:,zip])
zip_figure = plot(legend = false)
for uzip in unique_zips
    subs = filter_houses[filter_houses[:,zip].== uzip,: ]
    x = subs[:,latitude]
    y = subs[:,longitude]
    scatter!(zip_figure,x,y)
end
xlabel!("latitude")
ylabel!("longitude")
title!("Houses color-coded by zip code")
display(zip_figure)

```

✓ 0.1s

Julia

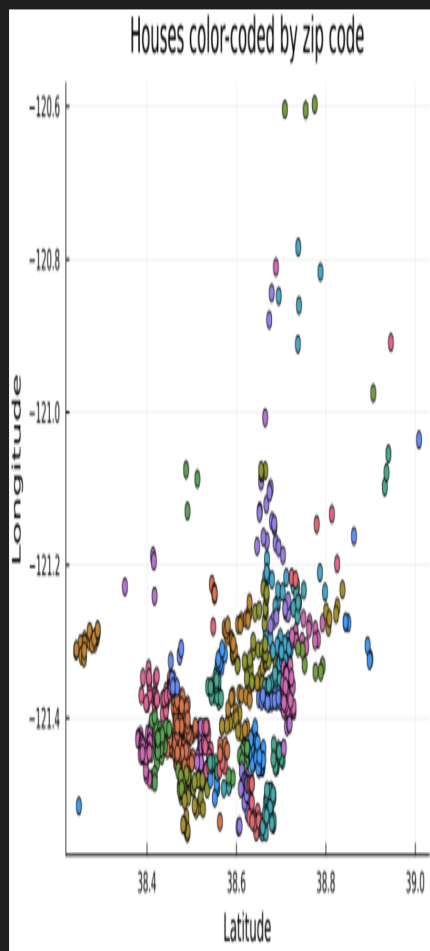


Рисунок 2.29: Построение графика с кластерами разных цветов по почтовому индексу

2.10 Кластеризация данных. Метод k ближайших соседей

Отообразим на графике соседей выбранного объекта недвижимости (рис. [fig:030]):


```

knearest = 10
id = 70
point = X[:,id]

# Поиск ближайших соседей:
kdtree = KDTree(X)
idxs, dists = knn(kdtree, point, knearest, true)

# Все объекты недвижимости:
x = filter_houses[:,latitude];
y = filter_houses[:,longitude];
scatter(x,y)
# Соседи:
x = filter_houses[idxs,:latitude];
y = filter_houses[idxs,:longitude];
scatter!(x,y)

```

✓ 0.1s

Julia

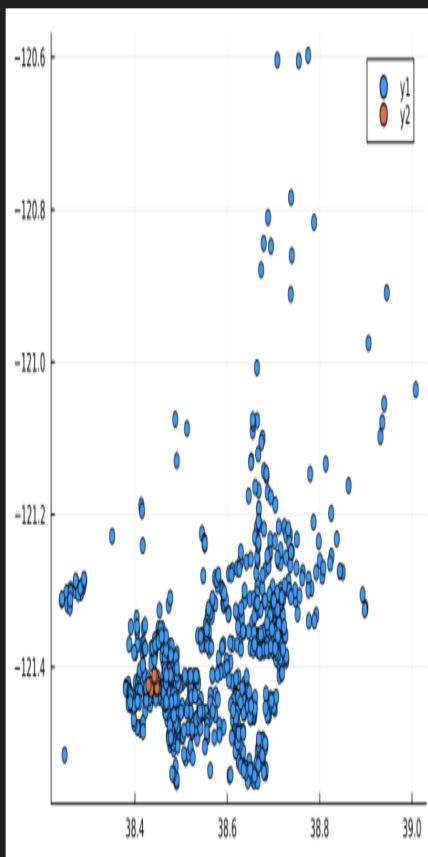


Рисунок 2.30: Отображение на графике соседей выбранного объекта недвижимости

Используя индексы `idxs` и функцию `:city` для индексации в DataFrame `filter_houses`, можно определить районы соседних домов (рис. [fig:031]):

```
# Фильтрация по районам соседних домов:
cities = filter_houses[idxs,:city]

✓ 0.5s Julia
```

```
10-element PooledArrays.PooledVector{String{15}, UInt32, Vector{UInt32}}:
"sacramento"
"ELK GROVE"
"sacramento"
"sacramento"
"sacramento"
"sacramento"
"ELK GROVE"
"ELK GROVE"
"ELK GROVE"
"ELK GROVE"
```

Рисунок 2.31: Определение районов соседних домов

2.11 Обработка данных. Метод главных компонент

Метод главных компонент (Principal Components Analysis, PCA) позволяет уменьшить размерность данных, потеряв наименьшее количество полезной информации. Метод имеет широкое применение в различных областях знаний, например, при визуализации данных, компрессии изображений, в эконометрике, некоторых гуманитарных предметных областях, например, в социологии или в политологии.

На примере с данными о недвижимости попробуем уменьшить размеры данных о цене и площади из набора данных домов (рис. [fig:032]):

```

# Фрейм с указанием площади и цены недвижимости:
F = filter_houses[1, [:sq_ft, :price]]
y = filter_houses[1, :price]
F = Matrix(F)'
y = y'

# Приведение типов данных к распределению для PCA:
M = fit(PCA, F)

# Выделение значений главных компонент в отдельную переменную:
Xr = reconstruct(M, y)

# Построение графика с выделением главных компонент:
scatter(F[1,:], F[2,:])
scatter!(Xr[1,: ], Xr[2,:])

```

✓ 28s

Julia

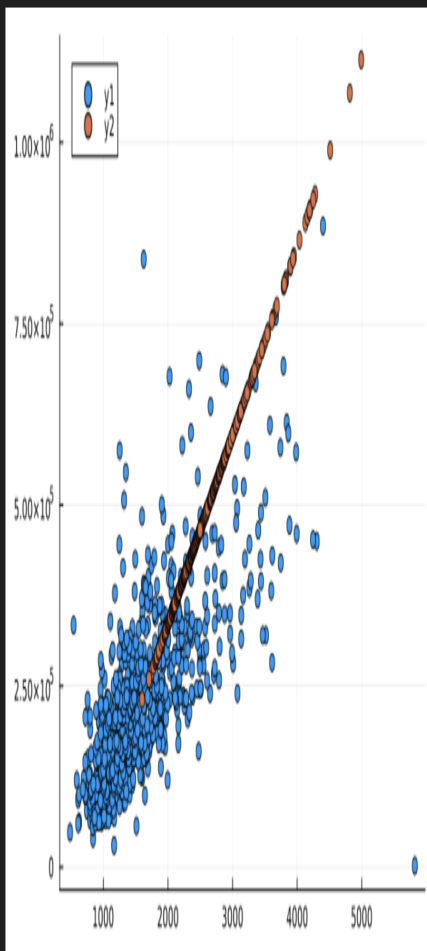


Рисунок 2.32: Попытка уменьшения размера данных о цене и площади из набора данных домов

2.12 Обработка данных. Линейная регрессия

Регрессионный анализ представляет собой набор статистических методов исследования влияния одной или нескольких независимых переменных (регрессоров) на зависимую (критериальная) переменную. Терминология зависимых и независимых переменных отражает лишь математическую зависимость переменных, а не причинноследственные отношения.

Наиболее распространённый вид регрессионного анализа — линейная регрессия, когда находят линейную функцию, которая согласно определённым математическим критериям наиболее соответствует данным.

Зададим случайный набор данных (можно использовать и полученные экспериментальным путём какие-то данные). Попробуем найти для данных лучшее соответствие (рис. [fig:033]):

```
xvals = repeat(1:0.5:10,inner=2)  
yvals = 3 + xvals + 2*rand(length(xvals)) - 1  
scatter(xvals,yvals,color=black,leg=false)
```

✓ 0.2s

Julia

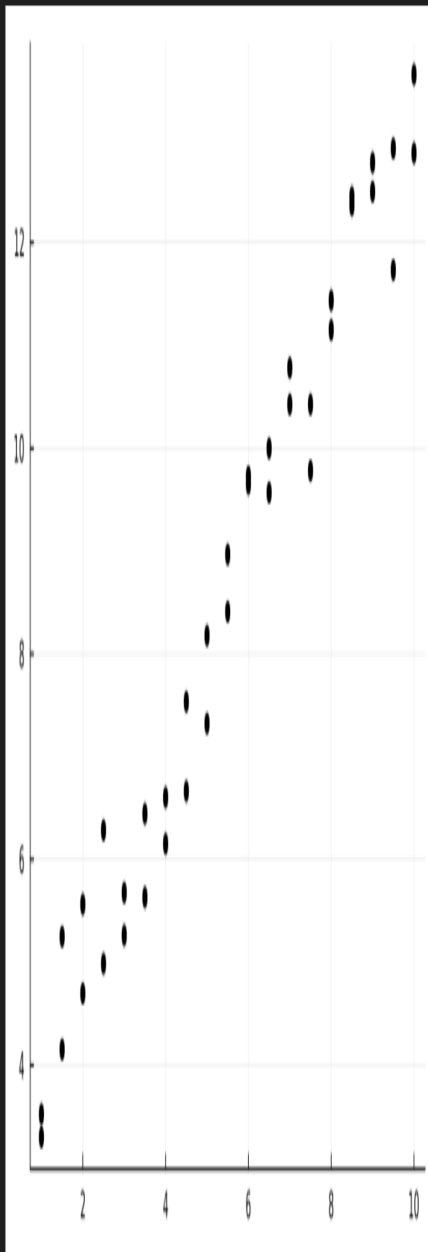


Рисунок 2.33: Исходные данные

Определим функцию линейной регрессии. Применим функцию линейной регрессии для построения соответствующего графика значений (рис. [fig:034]):


```

function find_best_fit(xvals,yvals)
    meanx = mean(xvals)
    meany = mean(yvals)
    stdx = std(xvals)
    stdy = std(yvals)
    r = cor(xvals,yvals)
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
end

```

✓ 0.0s

Julia

find_best_fit (generic function with 1 method)

```

a,b = find_best_fit(xvals,yvals)
ynew = a * xvals .+ b
plot!(xvals,ynew)

```

✓ 0.0s

Julia

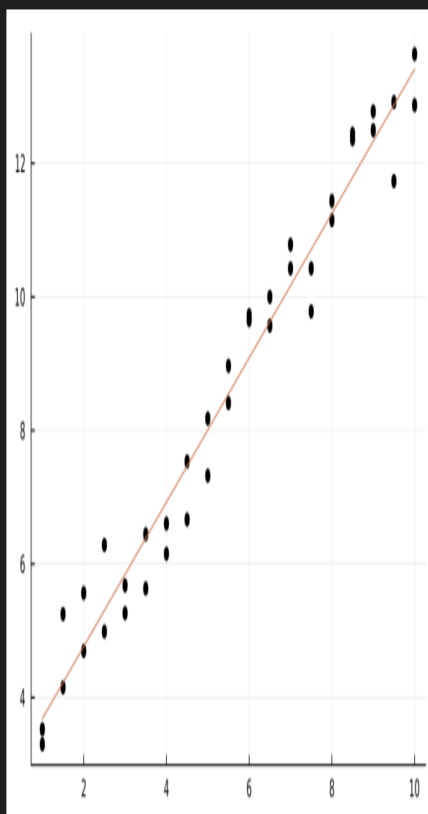


Рисунок 2.34: Применение функции для построения графика

Сгенерируем большой набор данных. Определим, сколько времени потребуется, чтобы найти соответствие этим данным. Для сравнения реализуем подобный код на языке Python. Используем пакет для анализа производительности, чтобы провести сравнение (рис. [fig:035] - рис. [fig:036]):

```
xvals = 1:100000;
xvals = repeat(xvals,inner=3);
yvals = 3 .* xvals + 2*rand(length(xvals)) .- 1;
@show size(xvals)
@show size(yvals)

✓ 0.0s Julia

size(xvals) = (300000,)
size(yvals) = (300000,)

(300000,)

@time a,b = find_best_fit(xvals,yvals)

✓ 0.3s Julia

0.023522 seconds (33.26 k allocations: 1.660 MiB, 95.95% compilation time)

(1.00000000095882504, 2.9993275259039365)

using PyCall
using Conda

✓ 0.0s Julia
```

Рисунок 2.35: Генерация данных. Определение времени соответствия

```
py"""
import numpy
def find_best_fit_python(xvals,yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals,yvals)[0][1]
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
"""

xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = py"find_best_fit_python"(xpy,ypy)

✓ 0.3s Julia

0.138421 seconds (138.93 k allocations: 6.838 MiB, 45.05% compilation time)

(1.00000000095882489, 2.999327525983972)

using BenchmarkTools
@btime a,b = py"find_best_fit_python"(xvals,yvals)
@btime a,b = find_best_fit(xvals,yvals)

✓ 256s Julia

4.182 ms (34 allocations: 960 bytes)
398.700 μs (1 allocation: 32 bytes)

(1.00000000095882504, 2.9993275259839365)
```

Рисунок 2.36: Сравнение

2.13 Самостоятельное выполнение

Загрузим

```
using RDatasets
iris = dataset("datasets", "iris")
```

Используем Clustering.jl для кластеризации на основе k-средних. Сделаем точечную диаграмму полученных кластеров. Для этого проиндексируем фрейм данных, преобразуем его в массив и транспонируем (рис. [fig:037] - рис. [fig:039]).

Задания

1.7) Кластеризация

```
using RDatasets, Clustering, Plots

iris = dataset("datasets", "iris")
iris_spec = Dict{String{<:AbstractString{<:ASCII}}, String{<:AbstractString{<:ASCII}}}
iris_spec["setosa"] = "setosa", "versicolour" = "versicolour", "virginica" = "virginica"
iris_spec["setosa"] = "setosa", "versicolour" = "versicolour", "virginica" = "virginica"

X = Matrix{Float64}(iris[1, :])
k = length(unique(iris[1, :]))
C = means(X, k)
df = DataFrame{Cluster = C.assignments, Sepallength = iris[1, :Sepallength],
  Sepallength = iris[1, :Sepallength], Petallength = iris[1, :Petallength],
  Petallength = iris[1, :Petallength], Species = iris[1, :Species]}

clusters_figure = plot(legend = false)
for i = 1:k
  clustered_houses = df[df[1, :Cluster] == i, :]
  xvals = clustered_houses[1, :Sepallength]
  yvals = clustered_houses[1, :Sepallength]
  scatter(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel("Sepallength")
ylabel("Sepallength")
title("Iris. Cluster")
display(clusters_figure)
```

Рисунок 2.37: Кластеризация

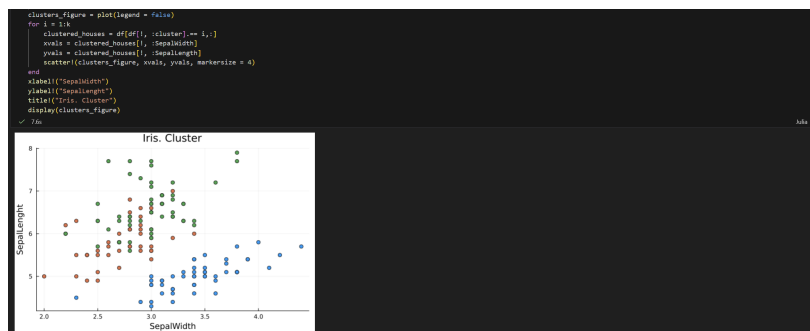


Рисунок 2.38: Кластеризация

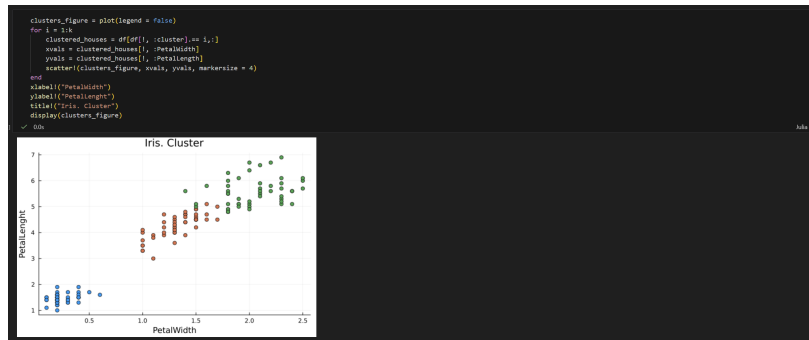


Рисунок 2.39: Кластеризация

Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ `randn(N, 3)`, массив результатов $N \times 1$, регрессионная зависимость является линейной. Найдем МНК-оценку для линейной модели (рис. [fig:040] - рис. [fig:041]).

- Сравним свои результаты с результатами использования `llsq` из `MultivariateStats.jl`.
- Сравним свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`.

Создадим матрицу данных X_2 , которая добавляет столбец единиц в начало матрицы данных, и решим систему линейных уравнений.

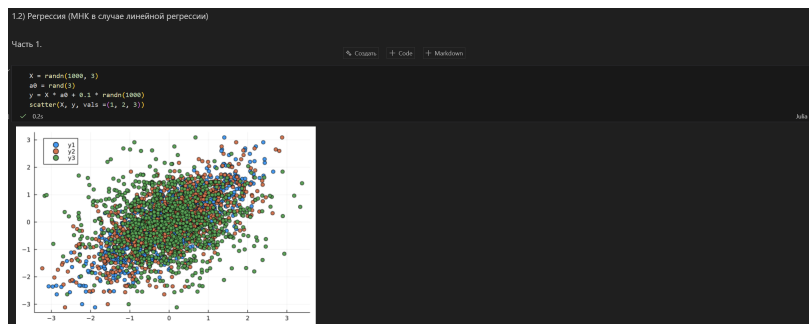


Рисунок 2.40: Регрессия

```

X2 = heat(ones(1000), X)
beta = (X2' * X2 \ (X2' * y))
✓ 10s

4-element Vector{Float64}:
 0.00233930646767834
 0.7906166934000253
 0.5000000000000224
 0.00638530618623160

using MultivariateStats
beta_obs = llog(X, y)
✓ 0s

4-element Vector{Float64}:
 0.7906166934000254
 0.5000000000000223
 0.00638530618623160
 0.002339306467678277

using GLM
df = DataFrame{heat{X, y}, :auto}
rename!(df, ["x1", "x2", "x3", "y"])
model = fit(GLMModel{y ~ x1 + x2 + x3}, df)
coef(model)
✓ 0s

4-element Vector{Float64}:
 0.00233930646767834
 0.7906166934000253
 0.5000000000000224
 0.00638530618623173

```

Рисунок 2.41: Регрессия

Найдем линию регрессии, используя данные (X, y) . Построим график (X, y) , используя точечный график. Добавим линию регрессии, используя `abline`!. Добавим заголовок «График регрессии» и подпишем оси x и y (рис. [fig:042]).

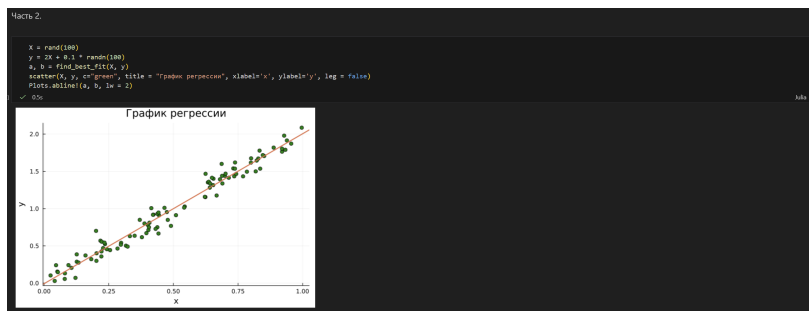


Рисунок 2.42: Регрессия

Построим траекторию возможных цен на акции:

- S – начальная цена акции;
- T – длина биномиального дерева в годах;
- n – количество периодов;
- $h = Tn$ – длина одного периода;
- σ – волатильность акции;
- r – годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;

$$p^* = \frac{\exp(rh) - d}{u - d};$$

Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуем построить траекторию курса акций(рис. [fig:043] - рис. [fig:044]).

```
1.3) Модель ценообразования биномиальных опционов
k.
S = 100.0      # начальная цена акции
T = 1.0        # время до истечения опциона
n = 10000      # количество шагов
sigma = 0.3    # волатильность
r = 0.08       # безрисковая процентная ставка
h = T / n      # длина шага времени

u = exp((r + sigma * sqrt(h))
d = exp((r - sigma * sqrt(h))
p = (exp(rh) - d) / (u - d)

function create_path(S, r, sigma, T, n)
    path = []
    S_ = S
    push!(path, S)
    for i in 1:n
        if rand() < p
            S_ *= u
        else
            S_ *= d
        end
        push!(path, S_)
    end
    return path
end

path = create_path(S, r, sigma, T, n)
plot(path, label="Stock Price Path", xlabel="Time Step", ylabel="Price", title="Stock Price Trajectory")
end
```

Рисунок 2.43: Модель ценообразования биномиальных опционов

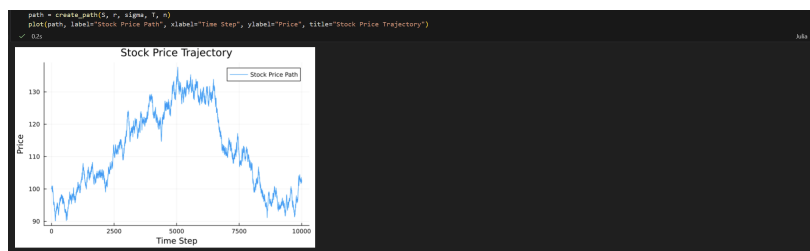


Рисунок 2.44: Модель ценообразования биномиальных опционов

Создадим функцию `createPath` (`S ::Float64`, `r ::Float64`, `sigma ::Float64`, `T ::Float64`, `n ::Int64`), которая создает траекторию цены акции с учетом начальных параметров. Используем `createPath`, чтобы создать 10 разных траекторий и построим их все на одном графике(рис. [fig:045] - рис. [fig:046]).


```

b.
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    V = Vector{Float64}()
    X = collect(0:T/n:1)
    u = exp(r + sigma * sqrt(h))
    d = exp(r - sigma * sqrt(h))
    p = (exp(r*h) - d) / (u - d)
    for i=0:n-1
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(V, S)
    end
    return(X, V)
end

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p
end

```

Рисунок 2.45: Модель ценообразования биномиальных опционов

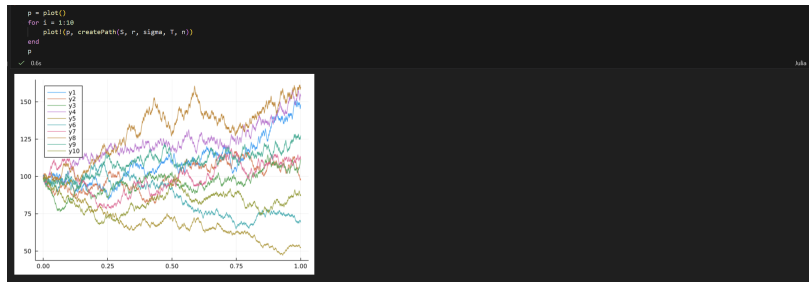


Рисунок 2.46: Модель ценообразования биномиальных опционов

Распараллелим генерацию траектории. Можем использовать `Threads.@threads`, `parmap` и `@parallel` (рис. [fig:047] - рис. [fig:048]).

```

c.
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    V = Vector{Float64}()
    X = collect(0:T/n:1)
    u = exp(r + sigma * sqrt(h))
    d = exp(r - sigma * sqrt(h))
    p = (exp(r*h) - d) / (u - d)
    @threads for i=0:n-1
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(V, S)
    end
    return(X, V)
end

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
p
end

```

Рисунок 2.47: Модель ценообразования биномиальных опционов

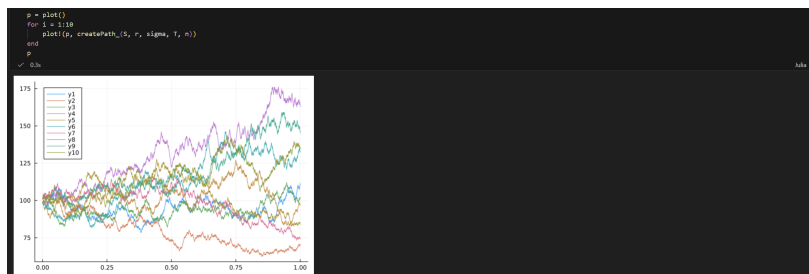


Рисунок 2.48: Модель ценообразования биномиальных опционов

3 Вывод

В ходе выполнения лабораторной работы я изучил специализированные пакеты Julia для обработки данных.

Список литературы

- [1] *Julia 1.11 Documentation*. URL: <https://docs.julialang.org/en/v1/>.
- [2] *JuliaLang*. URL: <https://julialang.org/>.